

Task 1 — Microservices Protocol

GitHub: https://github.com/Romko-studik/apz_homework_2/tree/micro_basics

1. Functional Correctness

POST Transactions

First deposit to "alice" (+500):

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/transaction" -ContentType "application/json" -Body '{"user_id": "alice", "amount": 500}'
>>
transaction_id      balance
-----
81ee4022-94f6-4c22-8173-ce232c96792d  500.0
```

Withdrawal for "alice" (-50):

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/transaction" -ContentType "application/json" -Body '{"user_id": "alice", "amount": -50}'
>>
transaction_id      balance
-----
0291378d-ba32-49d3-9503-91e726beba02  450.0
```

Deposit for "bob" (+200):

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/transaction" -ContentType "application/json" -Body '{"user_id": "bob", "amount": 200}'
>>
transaction_id      balance
-----
e4d00d15-a153-47d0-9cc8-02dce3415a7b  200.0
```

GET Requests

GET /user/alice — balance + transaction history:

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Uri "http://localhost:8000/user/alice"
>>
user_id balance transactions
-----
alice  450.0 @([transaction_id=81ee4022-94f6-4c22-8173-ce232c96792d; timestamp=1779312419.8015034; user_id=alice; amount=500.0], @(transaction_id=0291378d-ba32-49d3-9503-91e726beba02; timestamp=1779312422.6952002; user_id=alice; amount=-50.0)) 0.0075
```

GET /accounts — all balances:

```
PS D:\UNI_UCU\APZ\apz_homework_2> Invoke-RestMethod -Uri "http://localhost:8000/accounts"
>>
balances      query_time_seconds
-----
@{alice=450.0; bob=200.0} 0.0043
```

Service Console Output

facade-service logs:

```
[FACADE] POST tx=81ee4022-94f6-4c22-8173-ce232c96792d user=alice amount=+500.00 -> balance=500.00
INFO:      172.19.0.1:43888 - "POST /transaction HTTP/1.1" 201 Created
[FACADE] POST tx=0291378d-ba32-49d3-9503-91e726beba02 user=alice amount=-50.00 -> balance=450.00
INFO:      172.19.0.1:43888 - "POST /transaction HTTP/1.1" 201 Created
[FACADE] POST tx=e4d00d15-a153-47d0-9cc8-02dce3415a7b user=bob amount=+200.00 -> balance=200.00
INFO:      172.19.0.1:43888 - "POST /transaction HTTP/1.1" 201 Created
INFO:      172.19.0.1:42592 - "GET /user/alice HTTP/1.1" 200 OK
INFO:      172.19.0.1:42592 - "GET /accounts HTTP/1.1" 200 OK
```

logging-service logs:

```
[LOG] Stored transaction: {'transaction_id': '81ee4022-94f6-4c22-8173-ce232c96792d', 'timestamp': 1779312419.0815034, 'user_id': 'alice', 'amount': 500.0}
INFO:      172.19.0.4:41052 - "POST /log HTTP/1.1" 201 Created
[LOG] Stored transaction: {'transaction_id': '0291378d-ba32-49d3-9503-91e726beba02', 'timestamp': 1779312422.6952002, 'user_id': 'alice', 'amount': -50.0}
INFO:      172.19.0.4:41054 - "POST /log HTTP/1.1" 201 Created
[LOG] Stored transaction: {'transaction_id': 'e4d00d15-a153-47d0-9cc8-02dce3415a7b', 'timestamp': 1779312425.047201, 'user_id': 'bob', 'amount': 200.0}
```

counter-service logs:

```
[COUNTER] user=alice amount=+500.00 balance=500.00
INFO:      172.19.0.4:49224 - "POST /transaction HTTP/1.1" 200 OK
[COUNTER] user=alice amount=-50.00 balance=450.00
INFO:      172.19.0.4:49228 - "POST /transaction HTTP/1.1" 200 OK
[COUNTER] user=bob amount=+200.00 balance=200.00
INFO:      172.19.0.4:50846 - "POST /transaction HTTP/1.1" 200 OK
INFO:      172.19.0.4:50848 - "GET /balance/alice HTTP/1.1" 200 OK
INFO:      172.19.0.4:45150 - "GET /balances HTTP/1.1" 200 OK
```

2. Performance Testing

Scenario 1 — 10 clients × 10K transactions on own accounts

```
PS D:\UNI_UCU\APZ\apz_homework_2> python client/tests.py --scenario 1
>>

Scenario 1 – 10 clients x 10,000 transactions
Total time : 535.07s
Req/s      : 186.9
Logging svc – network avg: 5.37ms  processing avg: 0.040ms
Counter svc – network avg: 5.20ms  processing avg: 0.010ms

Correctness check:
  user_0: 10000.0 CORRECT
  user_1: 10000.0 CORRECT
  user_2: 10000.0 CORRECT
  user_3: 10000.0 CORRECT
  user_4: 10000.0 CORRECT
  user_5: 10000.0 CORRECT
  user_6: 10000.0 CORRECT
  user_7: 10000.0 CORRECT
  user_8: 10000.0 CORRECT
  user_9: 10000.0 CORRECT
```

Scenario 2 — 10 clients × 10K transactions on shared account

```
PS D:\UNI_UCU\APZ\apz_homework_2> python client/tests.py --scenario 2
>>

Scenario 2 – 10 clients x 10,000 transactions
Total time : 545.16s
Req/s      : 183.4
Logging svc – network avg: 5.84ms  processing avg: 0.040ms
Counter svc – network avg: 5.66ms  processing avg: 0.010ms

Correctness check:
  shared_user: 100000.0 CORRECT
```

3. Analysis

From our tests we can see that while each individual request runs quite quickly (~5ms network latency), when scaled to 100,000 requests (10 clients × 10,000 transactions each) the total test takes 535 seconds for scenario 1 and 545 seconds for scenario 2, netting ~187 and ~183 req/s respectively.

Counter-service is around 4x faster than logging-service at the processing level (0.01ms vs 0.04ms) since the counter only performs a simple arithmetic operation on a number, while logging-service stores a full transaction object into its hash map.

Since we measure request time both from facade's side and inside each service separately, we can clearly see that network overhead is the most time-consuming (~5ms per call) while

the actual service processing is negligible ($<0.05\text{ms}$). This means the bottleneck is Docker networking rather than service logic.

Scenario 1 and 2 perform almost identically despite all 10 clients writing to the same account in scenario 2. This shows that the `asyncio.Lock()` in counter-service introduces minimal time difference, and the shared account balance of 100,000 confirms no updates were lost.